

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

KONTROLA EKVIVALENTNÝCH ÚPRAV V
INTERAKTÍVNOM LOGICKOM PRACOVNOM
ZOŠITE
BAKALÁRSKA PRÁCA

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

KONTROLA EKVIVALENTNÝCH ÚPRAV V
INTERAKTÍVNOM LOGICKOM PRACOVNOM
ZOŠITE
BAKALÁRSKA PRÁCA

Študijný program: Aplikovaná Informatika
Študijný odbor: Informatika
Školiace pracovisko: Katedra informatiky
Školiteľ: Mgr. Ján Kluka, PhD.



Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Jakub Šimon Štofanič
Študijný program: aplikovaná informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Kontrola ekvivalentných úprav v interaktívnom logickom pracovnom zošite
Equivalent transformations checking in the interactive logic workbook

Anotácia: V rámci predmetu Logika pre informatikov sa využívajú ekvivalentné úpravy na konverziu formúl do konjunktívneho normálneho tvaru vo výrokovej logike. V spojení so skolemizáciou sa tiež využívajú na vytváranie ekvivalentného klauzálneho tvaru v prvorádovej logike, ktorý sa následne využíva v rezolvenčných dôkazoch. Študentstvo na cvičeniach z predmetu pracuje v elektronickom pracovnom hárku, ktorý vznikol v rámci predchádzajúcej bakalárskej práce (Mok, 2022). Je v ňom síce integrovaný nástroj na kontrolu rezolvenčných dôkazov (Jurík, 2020), kontrola predchádzajúcich ekvivalentných úprav a skolemizácie však chýba. Zadávanie komplexnejších cvičení, kde rezolvenca nadväzuje na takúto ekvivalentné úpravy a skolemizáciu je preto problematické a manuálna kontrola a hodnotenie úprav je veľmi prácna.

Cieľ: Vyvinúť nástroj na kontrolu ekvivalentných úprav a skolemizácie teórií v logike prvého rádu. Integrovať tento nástroj do logického pracovného hárku. Otestovať jeho použitie vo výučbe.

Literatúra: Mok, Matej. Interaktívny pracovný zošit pre výučbu logiky pre informatikov. Bakalárska práca. Bratislava : Univerzita Komenského, 2022.
Jurík, Norbert. Webový editor rezolvenčných dôkazov. Bakalárska práca. Bratislava : Univerzita Komenského, 2020.

Vedúci: Mgr. Ján Kľuka, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.
Dátum zadania: 08.10.2025

Dátum schválenia: 09.10.2025

doc. RNDr. Damas Gruska, PhD.
garant študijného programu

.....
študent

.....
vedúci práce

Abstrakt

V tejto práci sa venujeme vývoju webovej aplikácie, ktorá bude použitá ako učebná pomôcka. Aplikácia je určená na kontrolu ekvivalentných úprav logických formúl a skolemizácie, pomocou ktorých sa logické formuly prevádzajú do konjunktívnej normálnej formy. Pomocou prvorádového jazyka zadaného používateľom aplikácia skontroluje správnosť úprav formúl. Aplikácia bude kontrolovať iba samotné ekvivalentné úpravy a skolemizáciu, samotný postup je ponechaný na používateľa. Na vývoj aplikácie boli použité knižnice React a Redux. Aplikácia bude použitá ako učebná pomôcka pri výučbe predmetu Logika pre informatikov a bude integrovaná do interaktívneho logického zošita, ktorý sa na predmete využíva. Aplikácia bola úspešne testovaná v rámci spomenutého predmetu počas letného semestra 2025/2026.

Kľúčové slová: logika prvého rádu, Logic Workbook, prevod do konjunktívnej normálnej formy, CNF, webová aplikácia

Abstract

In this work, we are developing a web application that will be used as a teaching aid. The application is designed to check equivalent transformations of logical formulas and skolemization, with which logical formulas are converted to conjunctive normal form. Using the language of first-order logic entered by the user, the application will check the correctness of the formula transformations. The application will only check the equivalent transformations and skolemization themselves, the conversion to conjunctive normal form is left to the user. The React and Redux libraries were used to develop the application. The application will be used as a teaching aid in the teaching of the subject Logic for Computer Scientists and will be integrated into the interactive Logic Workbook that is used during the subject. The application was successfully tested within the aforementioned subject during the summer semester 2025/2026.

Keywords: first order logic, Logic Workbook, conversion to conjunctive normal form, CNF, web application

Obsah

Úvod	1
1 Teoretické znalosti	3
1.1 Logika prvého rádu a jej syntax	3
1.2 Ekvivalentné a ekvisplnitelné úpravy	7
1.3 Klauzálna teória	9
2 Východiská práce	11
2.1 Prehľad existujúceho ekosystému	11
2.2 Použité technológie	12
2.2.1 HTML, CSS, Bootstrap	12
2.2.2 TypeScript	12
2.2.3 React, React-Bootstrap	12
2.2.4 Redux	13
3 Požiadavky a návrh	14
3.1 Požiadavky	14
3.2 Návrh	15
3.2.1 Návrh stavu aplikácie	15
3.2.2 Návrh kontroly úprav	15
4 Implementácia	18
4.1 Súborová štruktúra	18
4.2 Rozšírenie modelu formúl a nástroja na syntaktickú analýzu	19
4.3 Kontrola ekvivalentných úprav	21
4.3.1 ImplicationEliminationChecker	21
4.3.2 ReorderingChecker	22
4.3.3 SkolemizationChecker	22
4.4 Implementácia používateľského rozhrania	23
4.5 Implementácia knižnice pre Logic Workbook	24

5	Testovanie	25
5.1	Spôsob testovania	25
5.2	Výsledky testovania	26
	Záver	28
	Príloha A: Zadanie domácej úlohy	31
	Príloha B: Testovací dotazník	32

Zoznam obrázkov

3.1	Diagram tried kontroly úprav	15
4.1	Ukážka používateľského rozhrania	24
5.1	Graf odpovedí na otázku Na aký účel ste kontrolór použili?	25
5.2	Graf odpovedí na otázku Akým spôsobom ste kontrolór použili?	26
5.3	Zadanie domácej úlohy	31

Úvod

Predmet Logika pre informatikou sa zaoberá logikou prvého rádu. Pri výučbe predmetu študenti používajú webový interaktívny logický pracovný zošit Logic Workbook, ktorý vznikol ako súčasť bakalárskej práce Mateja Moka [13]. Jednou z tém, ktoré sa vyučujú počas predmetu je dôkaz tvrdení pomocou rezolvenencie, pri čom sa využíva editor rezolvenčných dôkazov, ktorý bol vytvorený Norbertom Juríkom [11]. Na riešenie rezolvenencie je potrebný vstup (klauzálna teória), ktorý je vytvorený postupným aplikovaním ekvivalentných úprav a skolemizácie na počiatočné formuly. Logic Workbook neobsahuje nástroj, ktorý by kontroloval jednotlivé úpravy. Z toho vyplýva, že študenti musia robiť tento postup ručne, pričom ani študenti, ani vyučujúci nemajú ľahký spôsob kontroly správnosti úprav.

Cieľom práce je vytvoriť novú aplikáciu, ktorá bude integrovaná do Logic Workbooku. Aplikácia bude slúžiť na kontrolu správneho prevodu formuly do konjunktívnej normálnej formy. Keďže je určená ako učebná pomôcka, aplikácia bude kontrolovať iba správnosť jednotlivých ekvivalentných úprav a skolemizácie. Ak by prevod bol príliš automatizovaný mohla by nastať situácia, kde študenti nebudú rozumieť postupu, ale iba dosiahnu správny výsledok spôsobom pokus-omyl. Preto samotný postup prevodu bude ponechaný na študentov. Aplikácia umožní vybrať jeden druh ekvivalentnej úpravy, ktorá sa v danom kroku uplatní buď na počiatočnú formulu alebo formulu vytvorenú predošlou ekvivalentnou úpravou. Aplikácia v každom kroku skontroluje či sú dve formuly ekvivalentné podľa vybranej úpravy. Pomocou postupností týchto úprav, študenti budú môcť skontrolovať svoj postup. Keďže študenti často potrebujú previesť viacero formúl do konjunktívnej normálnej formy, budú môcť aplikácii zadať viacero počiatočných formúl. Každá počiatočná formula bude začiatkom osobitnej postupnosti ekvivalentných úprav. Keďže aplikácia bude integrovaná do logického pracovného zošita, jej používateľské rozhranie bude s ním kompatibilné.

V prvej kapitole zhrnieme teoretické znalosti. V druhej kapitole krátko opíšeme existujúci ekosystém a zhrnieme technológie, ktoré sme použili pri tvorbe aplikácie. Potom v tretej kapitole sa budeme venovať návrhu aplikácie, jej internému stavu, základnému návrhu kontroly ekvivalentných úprav a tiež zhrnieme požiadavky pre aplikáciu. V štvrtej kapitole bližšie popíšeme samotnú implementáciu aplikácie, jej súborovú štruktúru a používateľské rozhranie. Tiež opíšeme viacero špecifických implementácií kontroly

správnosti úprav. Nakoniec v piatej a poslednej kapitole popíšeme ako sme aplikáciu testovali a zhrnieme výsledky testovania.

Kapitola 1

Teoretické znalosti

V tejto kapitole zhrnieme definície potrebných pojmov. Definície sú prebraté z prednášok z Logiky pre informatikov [12]. Budeme používať skratku vtt, ktorá znamená „vtedy a práve vtedy keď“.

1.1 Logika prvého rádu a jej syntax

Logika prvého rádu je rodina jazykov, ktoré umožňujú opisovať vlastnosti a vzťahy objektov z určitej domény pomocou logických spojok a kvantifikácie cez tieto objekty. Jazyky z tejto rodiny zdieľajú logické a pomocné symboly, ale líšia sa v mimologických symboloch. Mimologické symboly opisujú pravé vlastnosti alebo vzťahy medzi objektmi.

Symbolmi jazyka logiky prvého rádu $\mathcal{L}$ sú mimologické, logické a pomocné symboly a individuové premenné. *Individuové premenné* sú z nejakej nekonečnej spočítateľnej množiny $\mathcal{V}_{\mathcal{L}}$. *Mimologické symboly* sa delia na funkčné a predikátové symboly a individuové konštanty, ktoré sú v danom poradí z nejakých spočítateľných množín $\mathcal{F}_{\mathcal{L}}$, $\mathcal{P}_{\mathcal{L}}$ a $\mathcal{C}_{\mathcal{L}}$. *Logické symboly* sa delia na *logické spojky* (unárna $\neg$ a binárne $\wedge, \vee, \rightarrow$), *symbol rovnosti* ($\doteq$) a *kvantifikátory* (existenčný $\exists$ a všeobecný $\forall$). *Pomocné symboly* sú $(,)$ a $,$ (ľavá zátvorka, pravá zátvorka a čiarka). Množiny $\mathcal{V}_{\mathcal{L}}, \mathcal{C}_{\mathcal{L}}, \mathcal{F}_{\mathcal{L}}, \mathcal{P}_{\mathcal{L}}$ sú vzájomne disjunktné. Logické ani pomocné symboly sa nevyskytujú v symboloch z $\mathcal{V}_{\mathcal{L}}, \mathcal{C}_{\mathcal{L}}, \mathcal{F}_{\mathcal{L}}, \mathcal{P}_{\mathcal{L}}$. Každému symbolu $s \in \mathcal{P}_{\mathcal{L}} \cup \mathcal{F}_{\mathcal{L}}$ je priradená arita $\text{ar}(s) \in \mathbb{N}^+$. Napríklad $\mathcal{V}_{\mathcal{L}_p} = \{x, y, z\}$, $\mathcal{C}_{\mathcal{L}_p} = \{\text{kitty}\}$, $\mathcal{F}_{\mathcal{L}_p} = \{\text{father}^1\}$, $\mathcal{P}_{\mathcal{L}_p} = \{\text{cat}^1, \text{loves}^2\}$

Na označenie objektov jazyka slúžia *termy*. Množina $\mathcal{T}_{\mathcal{L}}$ *termov* jazyka logiky prvého rádu $\mathcal{L}$ je najmenšia množina postupností symbolov jazyka $\mathcal{L}$, pre ktorú platí $\mathcal{V}_{\mathcal{L}} \subseteq \mathcal{T}_{\mathcal{L}}$, $\mathcal{C}_{\mathcal{L}} \subseteq \mathcal{T}_{\mathcal{L}}$ a ak f je funkčný symbol s aritou n a $t_1, \dots, t_n$ patria do $\mathcal{T}_{\mathcal{L}}$, tak aj postupnosť symbolov $f(t_1, \dots, t_n) \in \mathcal{T}_{\mathcal{L}}$. Každý prvok $\mathcal{T}_{\mathcal{L}}$ je *term* jazyka $\mathcal{L}$ a nič iné nie je termom jazyka $\mathcal{L}$. Napríklad $\mathcal{T}_{\mathcal{L}_p} = \{x, y, z, \text{kitty}, \text{father}(x), \text{father}(y), \text{father}(z), \text{father}(\text{kitty}), \text{father}(\text{father}(x)), \text{father}(\text{father}(y)) \dots\}$

Rovnostný atóm jazyka $\mathcal{L}$ je každá postupnosť symbolov $t_1 \doteq t_2$, kde t_1 a t_2 sú termy jazyka $\mathcal{L}$. *Predikátový atóm* jazyka $\mathcal{L}$ je každá postupnosť symbolov $P(t_1, \dots, t_n)$, kde P je predikátový symbol s aritou n a $t_1, \dots, t_n$ sú termy jazyka $\mathcal{L}$. *Atomickými formulami* (skrátene *atómami*) jazyka $\mathcal{L}$ súhrnne nazývame všetky rovnostné a predikátové atómy jazyka $\mathcal{L}$. Atómy zodpovedajú jednoduchým výrokom o objektoch jazyka, napríklad „ x je totožný s y “ alebo „ z je mačka“. Množinu všetkých *atómov* jazyka $\mathcal{L}$ označujeme $\mathcal{A}_{\mathcal{L}}$. Napríklad $\mathcal{A}_{\mathcal{L}_p} = \{x \doteq x, x \doteq y, x \doteq \text{father}(x), \dots, \text{cat}(x), \text{cat}(y), \dots, \text{loves}(x, x), \text{loves}(x, z), \text{loves}(x, \text{father}(x)), \dots\}$

Množina $\mathcal{E}_{\mathcal{L}}$ všetkých *formúl* jazyka logiky prvého rádu $\mathcal{L}$ je najmenšia množina postupností symbolov jazyka $\mathcal{L}$, pre ktorú platí:

1. $\mathcal{A}_{\mathcal{L}} \subseteq \mathcal{E}_{\mathcal{L}}$,
2. Ak A patrí do $\mathcal{E}_{\mathcal{L}}$, tak aj postupnosť symbolov $\neg A$ patrí do $\mathcal{E}_{\mathcal{L}}$ a nazývame ju *negácia* formuly A .
3. Ak A a B sú v $\mathcal{E}_{\mathcal{L}}$, tak aj postupnosti symbolov $(A \wedge B)$, $(A \vee B)$ a $(A \rightarrow B)$ patria do $\mathcal{E}_{\mathcal{L}}$ a nazývame ich postupne *konjunkcia*, *disjunkcia* a *implikácia* formúl A a B .
4. Ak x je individuová premenná a A patrí do $\mathcal{E}_{\mathcal{L}}$, tak aj postupnosti symbolov $\exists x A$ a $\forall x A$ patria do $\mathcal{E}_{\mathcal{L}}$ a nazývame ich postupne *existenčná* a *všeobecná kvantifikácia* formuly A vzhľadom na x .

Každý prvok A množiny $\mathcal{E}_{\mathcal{L}}$ nazývame *formulou* jazyka $\mathcal{L}$.

Nech A je postupnosť symbolov, nech B je formula, nech $Q \in \{\forall, \exists\}$ nech x je premenná. V postupnosti $A = \dots QxB \dots$ sa výskyt formuly QxB nazýva *oblasť platnosti kvantifikátora* Qx v A . Výskyt premennej x v A je *viazaný* vtt sa nachádza v niektorej oblasti platnosti kvantifikátora $\forall x$ alebo $\exists x$ v A . Výskyt premennej x v A je *voľný* vtt sa nenachádza v žiadnej oblasti platnosti kvantifikátora $\forall x$ ani $\exists x$ v A . Premenná x je *viazaná* v A vtt x sa vyskytuje v A a všetky výskyty x v A sú viazané. Premenná x je *voľná* v A vtt x má v A aspoň jeden voľný výskyt. Množinu voľných premenných formuly A označíme $\text{free}(A)$. Pre každú individuovú premennú x , každý symbol konštanty a , každú aritu $n > 0$, každý predikátový symbol P s aritou n , všetky

termy $t_1, \dots, t_n$ a všetky formuly A, B platí:

$$\begin{aligned}
 \text{free}(x) &= \{x\} \\
 \text{free}(a) &= \{\} \\
 \text{free}(t_1 \doteq t_2) &= \text{free}(t_1) \cup \text{free}(t_2) \\
 \text{free}(P(t_1, \dots, t_n)) &= \text{free}(t_1) \cup \dots \cup \text{free}(t_n) \\
 \text{free}(\neg A) &= \text{free}(A) \\
 \text{free}((A \wedge B)) &= \text{free}((A \vee B)) = \text{free}((A \rightarrow B)) = \\
 &= \text{free}(A) \cup \text{free}(B) \\
 \text{free}(\forall x A) &= \text{free}(\exists x A) = \text{free}(A) \setminus \{x\}
 \end{aligned}$$

Formula A jazyka $\mathcal{L}$ je *uzavretá* vtt žiadna premenná nie je voľná v A (teda $\text{free}(A) = \emptyset$). Opak uzavretej formuly je *otvorená* formula. *Uzavreté* formuly zodpovedajú výrokom a *otvorené* formuly zodpovedajú výrokovým formám.

Napríklad pre jazyk $\mathcal{L}_p$ s $\mathcal{C}_{\mathcal{L}_p} = \{\text{kitty}\}$, $\mathcal{F}_{\mathcal{L}_p} = \{\text{father}^1\}$, $\mathcal{P}_{\mathcal{L}_p} = \{\text{cat}^1, \text{loves}^2\}$ je

$$X = \exists x \forall y (\text{cat}(y) \rightarrow \text{loves}(x, y)) \wedge \forall x ((\text{loves}(x, \text{kitty})) \rightarrow \text{loves}(\text{father}(\text{kitty}), x))$$

formulou. Nazvime ju X . Neformálne znenie výroku, ktorý zodpovedá X je „Niečo miluje všetky mačky, a zároveň ak niečo miluje kitty, tak ho otec kitty tiež miluje“. Termy použité v X sú y, x , kitty a $\text{father}(\text{kitty})$. Atómy použité v X sú $\text{cat}(y)$, $\text{loves}(x, y)$, $\text{loves}(x, \text{kitty})$ a $\text{loves}(\text{father}(\text{kitty}), x)$. Oblasťou platnosti kvantifikátora $\forall y$ v X je $\forall y (\text{cat}(y) \rightarrow \text{loves}(x, y))$, čo je tiež formula. Nazvime ju Y . X je uzavretá, lebo $\text{free}(X) = \emptyset$ ale Y nie je, lebo $\text{free}(Y) = \{x\}$.

Na priradenie významu mimlogickým symbolom jazyka, sa používa *štruktúra*. *Štruktúrou* pre jazyk $\mathcal{L}$ nazývame každú dvojicu $\mathcal{M} = (D, i)$, kde *doména* D štruktúry $\mathcal{M}$ je ľubovoľná neprázdna množina; *interpretačná funkcia* i štruktúry $\mathcal{M}$ je zobrazenie, ktoré

- každej individuovej konštante c jazyka $\mathcal{L}$ priraduje prvok $i(c) \in D$
- každému funkčnému symbolu f jazyka $\mathcal{L}$ s aritou n priraduje funkciu $i(f) : D^n \rightarrow D$
- každému predikátovému symbolu P jazyka $\mathcal{L}$ s aritou n priraduje množinu $i(P) \subseteq D^n$.

Napríklad $\mathcal{M}_p = (D, i)$ je štruktúra pre $\mathcal{L}_p$, kde $D = \{a, b, c, d, e\}$ a

- $i(\text{kitty}) = d$
- $i(\text{father}) = \{(d, e), (a, b), (b, c), (c, c), (e, e)\}$

- $i(\text{cat}) = \{b, c, d, e\}$
- $i(\text{loves}) = \{(a, b), (b, c), (c, d), (d, e), (e, a), (e, d)\}$

Nech $\mathcal{M} = (D, i)$ je štruktúra pre jazyk $\mathcal{L}$. *Ohodnotenie indivíduových premenných* je ľubovoľná funkcia $e : \mathcal{V}_{\mathcal{L}} \rightarrow D$. Nech ďalej x je indivíduová premenná z $\mathcal{V}_{\mathcal{L}}$ a v je prvok D . Zápis $e(x/v)$ označuje ohodnotenie e' indivíduových premenných, pre ktoré platí $e'(x) = v$ a $e'(y) = e(y)$, ak y je iná premenná ako x .

Nech $\mathcal{M} = (D, i)$ je štruktúra pre jazyk logiky prvého rádu $\mathcal{L}$, nech e je ohodnotenie premenných. *Hodnotou termu* t v štruktúre $\mathcal{M}$ pri ohodnotení premenných e je prvok z D označovaný $t^{\mathcal{M}}[e]$ a zadaný indukčne pre všetky premenné x , konštanty a , každú aritu n , všetky funkčné symboly f s aritou n , a všetky termy $t_1, \dots, t_n$ nasledovne:

$$\begin{aligned} x^{\mathcal{M}}[e] &= e(x) \\ a^{\mathcal{M}}[e] &= i(a) \\ (f(t_1, \dots, t_n))^{\mathcal{M}}[e] &= i(f)(t_1^{\mathcal{M}}[e], \dots, t_n^{\mathcal{M}}[e]). \end{aligned}$$

Napríklad pre štruktúru $\mathcal{M}_p$ a ohodnotenie $e_p = \{(x, a), (y, b), (z, c)\}$ platí

$$\begin{aligned} x^{\mathcal{M}_p}[e_p] &= e_p(x) = a \\ \text{kitty}^{\mathcal{M}_p}[e_p] &= i(\text{kitty}) = d \\ (\text{father}(\text{kitty}))^{\mathcal{M}_p}[e_p] &= i(\text{father})(\text{kitty}^{\mathcal{M}_p}[e_p]) = e \end{aligned}$$

Nech $\mathcal{M} = (D, i)$ je štruktúra pre $\mathcal{L}$, e je ohodnotenie premenných. Relácia *štruktúra $\mathcal{M}$ spĺňa formulu X pri ohodnotení e* ($\mathcal{M} \models X[e]$) má nasledovnú indukčnú definíciu:

$$\begin{aligned} \mathcal{M} \models t_1 \doteq t_2[e] &\text{ vtt } t_1^{\mathcal{M}}[e] = t_2^{\mathcal{M}}[e] \\ \mathcal{M} \models P(t_1, \dots, t_n)[e] &\text{ vtt } (t_1^{\mathcal{M}}[e], \dots, t_n^{\mathcal{M}}[e]) \in i(P) \\ \mathcal{M} \models \neg A[e] &\text{ vtt } \mathcal{M} \not\models A[e] \\ \mathcal{M} \models (A \wedge B)[e] &\text{ vtt } \mathcal{M} \models A[e] \text{ a zároveň } \mathcal{M} \models B[e] \\ \mathcal{M} \models (A \vee B)[e] &\text{ vtt } \mathcal{M} \models A[e] \text{ alebo } \mathcal{M} \models B[e] \\ \mathcal{M} \models (A \rightarrow B)[e] &\text{ vtt } \mathcal{M} \not\models A[e] \text{ alebo } \mathcal{M} \models B[e] \\ \mathcal{M} \models \exists x A[e] &\text{ vtt pre nejaký prvok } m \in D \text{ máme } \mathcal{M} \models A[e(x/m)] \\ \mathcal{M} \models \forall x A[e] &\text{ vtt pre každý prvok } m \in D \text{ máme } \mathcal{M} \models A[e(x/m)] \end{aligned}$$

pre všetky arity $n > 0$, všetky predikátové symboly P s aritou n , všetky termy $t_1, \dots, t_n$, všetky premenné x a všetky formuly A, B jazyka $\mathcal{L}$.

Nech X je uzavretá formula jazyka $\mathcal{L}$ a nech $\mathcal{M}$ je štruktúra pre $\mathcal{L}$. Formula X je *pravdivá* v štruktúre $\mathcal{M}$ (skrátene $\mathcal{M} \models X$) vtt $\mathcal{M}$ spĺňa formulu X pri každom ohodnotení e . Vtedy tiež hovoríme, že $\mathcal{M}$ je *modelom* formuly X . Model množiny formúl T je štruktúra, v ktorej sú pravdivé všetky formuly z T .

Napríklad $\mathcal{M}_p$ je modelom $X = \exists x \text{ loves}(x, \text{kitty})$, pretože: nech e je ľubovoľné ohodnotenie, potom

- $\mathcal{M}_p \models \exists x \text{ loves}(x, \text{kitty})[e]$ vtt
- $\mathcal{M}_p \models \text{loves}(x, \text{kitty})[e(x/a)]$,
- $(x^{\mathcal{M}_p}[e(x/a)], \text{kitty}^{\mathcal{M}_p}[e(x/a)]) \in i(\text{loves})$
- $(a, i(\text{kitty})) \in i(\text{loves})$
- $(a, d) \in i(\text{loves})$

1.2 Ekvivalentné a ekvisplniteľné úpravy

Rozhodnúť o pravdivosti alebo vlastnostiach niektorých formúl je oveľa ťažšie ako iných. Preto je užitočné ich zmeniť do formy, s ktorou sa pracuje ľahšie. Na to slúžia ekvivalentné úpravy. Pomocou nich vieme formulu zmeniť na inú formulu, ktorá má tie isté vlastnosti.

Formula X je *prvorádovo ekvivalentná* s formulou Y ($X \Leftrightarrow Y$) vtt pre každú štruktúru $\mathcal{M}$ a každé ohodnotenie e máme $\mathcal{M} \models X[e]$ vtt $\mathcal{M} \models Y[e]$. Príslovku „prvorádovo“ budeme zvyčajne vynechávať. Množiny formúl S a T sú (prvorádovo) *ekvisplniteľné* (rovnako splniteľné) vtt S má model práve vtedy, keď T má model.

Nech $\mathcal{L}$ je jazyk logiky prvého rádu a nech X, A, B sú formuly jazyka $\mathcal{L}$ také, že $\text{free}(A) \supseteq \text{free}(B)$. *Substitúciou* B za A v X (skrátene $X[A|B]$) nazývame formulu, ktorá vznikne nahradením každého výskytu A v X formulou B . Pre všetky formuly A, B, X, Y jazyka $\mathcal{L}$, všetky individuové premenné x , všetky kvantifikátory $Q \in \{\forall, \exists\}$ a všetky binárne spojky $b \in \{\wedge, \vee, \rightarrow\}$ platí

- $X[A|B] = B$, ak $A = X$
- $X[A|B] = X$, ak X je atóm a $A \neq X$
- $(\neg X)[A|B] = \neg(X[A|B])$, ak $A \neq \neg X$
- $(XbY)[A|B] = ((X[A|B])b(Y[A|B]))$, ak $A \neq (XbY)$
- $(QxX)[A|B] = Qx(X[A|B])$, ak $A \neq (QxX)$

Nech $\mathcal{L}$ je jazyk logiky prvého rádu a nech X je formula, A a B sú výrokovologicky ekvivalentné formuly jazyka $\mathcal{L}$. Potom formuly X a $X[A|B]$ sú tiež výrokovologicky ekvivalentné.

Katalóg ekvivalentných úprav Pri ekvivalentných úpravách sa používa viacero známych dvojíc ekvivalentných formúl. Tieto dvojice vyjadrujú rôzne vlastnosti logických

spojok, ako napríklad komutatívnosť, distributívnosť. Ďalej uvádzame ich zoznam. Pre ľubovoľnú tautológiu $\top$, ľubovoľnú nespítniteľnú formulu $\perp$, všetky formuly A, B, C , každú premennú x :

$$(A \rightarrow B) \Leftrightarrow (\neg A \vee B) \quad (A \leftrightarrow B) \Leftrightarrow (A \rightarrow B) \wedge (B \rightarrow A) \quad (1.1)$$

$$\neg(A \wedge B) \Leftrightarrow (\neg A \vee \neg B) \quad \neg(A \vee B) \Leftrightarrow (\neg A \wedge \neg B) \quad (1.2)$$

$$\neg\top \Leftrightarrow \perp \quad \neg\perp \Leftrightarrow \top \quad (1.3)$$

$$\neg\neg A \Leftrightarrow A \quad (1.4)$$

$$\neg\exists x A \Leftrightarrow \forall x \neg A \quad \neg\forall x A \Leftrightarrow \exists x \neg A \quad (1.5)$$

$$(A \wedge (B \vee C)) \Leftrightarrow ((A \wedge B) \vee (A \wedge C)) \quad (A \vee (B \wedge C)) \Leftrightarrow ((A \vee B) \wedge (A \vee C)) \quad (1.6)$$

$$(A \wedge (B \wedge C)) \Leftrightarrow ((A \wedge B) \wedge C) \quad (A \vee (B \vee C)) \Leftrightarrow ((A \vee B) \vee C) \quad (1.7)$$

$$(A \wedge B) \Leftrightarrow (B \wedge A) \quad (A \vee B) \Leftrightarrow (B \vee A) \quad (1.8)$$

$$(A \wedge \top) \Leftrightarrow A \quad (A \vee \perp) \Leftrightarrow A \quad (1.9)$$

$$(A \wedge A) \Leftrightarrow A \quad (A \vee A) \Leftrightarrow A \quad (1.10)$$

$$(A \vee (A \wedge B)) \Leftrightarrow A \quad (A \wedge (A \vee B)) \Leftrightarrow A \quad (1.11)$$

$$(A \wedge \perp) \Leftrightarrow \perp \quad (A \vee \top) \Leftrightarrow \top \quad (1.12)$$

$$(A \wedge \neg A) \Leftrightarrow \perp \quad (A \vee \neg A) \Leftrightarrow \top \quad (1.13)$$

$$\exists x(A \vee B) \Leftrightarrow (\exists x A \vee \exists x B) \quad \forall x(A \wedge B) \Leftrightarrow (\forall x A \wedge \forall x B) \quad (1.14)$$

Pre každú formulu D , v ktorej sa x nevyskytuje voľná:

$$\exists x(A \vee D) \Leftrightarrow \exists x A \vee D \quad \forall x(A \vee D) \Leftrightarrow \forall x A \vee D \quad (1.15)$$

$$\exists x(A \wedge D) \Leftrightarrow \exists x A \wedge D \quad \forall x(A \wedge D) \Leftrightarrow \forall x A \wedge D \quad (1.16)$$

$$\exists x D \Leftrightarrow D \quad \forall x D \Leftrightarrow D \quad (1.17)$$

Za podmienky, že y nemá voľný výskyt v A :

$$\exists x A \Leftrightarrow \exists y A\{x \mapsto y\} \quad \forall x A \Leftrightarrow \forall y A\{x \mapsto y\} \quad (1.18)$$

Substitúciou (v jazyku $\mathcal{L}$) nazývame každé zobrazenie $\sigma : V \rightarrow \mathcal{T}_{\mathcal{L}}$ z nejakej množiny individuových premenných $V \subseteq \mathcal{V}_{\mathcal{L}}$ do termov jazyka $\mathcal{L}$. Nech A je postupnosť symbolov (term alebo formula), nech $t, t_1, \dots, t_n$ sú termy a $x, x_1, \dots, x_n$ sú premenné. Term t je *substituovateľný* za premennú x v A vtt nie je pravda, že pre niektorú premennú y vyskytujúcu sa v t platí, že v nejakej oblasti platnosti kvantifikátora $\exists y$ alebo $\forall y$ vo formule A sa premenná x vyskytuje voľná. Substitúcia $\{x_1 \mapsto t_1, \dots, x_n \mapsto t_n\}$ je *aplikovateľná* na A vtt term t_i je substituovateľný za x_i v A pre každé $i \in \{1, \dots, n\}$. Nech A je postupnosť symbolov, nech $\sigma = \{x_1 \mapsto t_1, \dots, x_n \mapsto t_n\}$ je substitúcia. Ak σ je aplikovateľná na A , tak $A\sigma$ je postupnosť symbolov, ktorá vznikne súčasným nahradením každého voľného výskytu premennej x_i v A termom t_i . Nech $\mathcal{M}$ je štruktúra pre

jazyk $\mathcal{L}$, e ohodnotenie individuových premenných, nech $\sigma = \{x_1 \mapsto t_1, \dots, x_n \mapsto t_n\}$ je substitúcia, nech t je term jazyka $\mathcal{L}$ a nech A je formula jazyka $\mathcal{L}$ a σ je aplikovateľná na A . Potom $(t\sigma)^{\mathcal{M}}[e] = t^{\mathcal{M}}[e(x_1/t_1^{\mathcal{M}}[e])\dots(x_n/t_n^{\mathcal{M}}[e])]$ a $\mathcal{M} \models A\sigma[e]$ vtt $\mathcal{M} \models A[e(x_1/t_1^{\mathcal{M}}[e])\dots(x_n/t_n^{\mathcal{M}}[e])]$.

1.3 Klauzálna teória

Jednou metódou dôkazu pravdivosti tvrdenia v logika prvého rádu je rezolvenca. Na riešenie rezolvenacie je potrebný vstup vo forme klauzálnnej teórie. *Literál* je atomická formula $P(t_1, \dots, t_m)$ jazyka $\mathcal{L}$ alebo jej negácia $\neg P(t_1, \dots, t_m)$. *Klauzula* je všeobecný uzáver disjunkcie literálov, teda uzavretá formula jazyka $\mathcal{L}$ v tvare $\forall x_1 \dots \forall x_k (L_1 \vee \dots \vee L_n)$ kde $L_1, \dots, L_n$ sú literály a $x_1, \dots, x_k$ sú všetky voľné premenné formuly $L_1 \vee \dots \vee L_n$. Klauzula môže byť aj jednotková ($\forall x_1 \dots \forall x_k L_1$) alebo prázdna ($\square$). *Klauzálna teória* je množina klauzúl $\{C_1, \dots, C_n\}$. Môže byť tvorená aj jedinou klauzulou alebo byť prázdna.

Formula X je v *negačnom normálnom tvare* (NNF) vtt neobsahuje implikáciu a pre každú jej podformulu $\neg A$ platí, že A je atomická formula. Formula X je v *prenexnom normálnom tvare* (PNF) vtt má tvar $Q_1 x_1 \dots Q_n x_n A$, kde $Q_i \in \{\forall, \exists\}$, x_i je premenná a A je formula bez kvantifikátorov. Formula X je v *konjunktívnom normálnom tvare* (CNF) vtt má tvar $\forall x_1 \dots \forall x_n A$, kde x_i je premenná a A je konjunkcia disjunkcií literálov.

Skolemizácia je úprava formuly X v NNF, ktorou nahradíme existenčné kvantifikátory novými konštantami alebo funkčnými symbolmi. Ak sa vo formule X vyskytuje $\exists y A$ mimo všetkých oblastí platnosti všeobecných kvantifikátorov, tak pridáme do jazyka novú skolemovskú konštantu c a každý výskyt podformuly $\exists y A$ v X mimo všetkých oblastí platnosti všeobecných kvantifikátorov nahradíme formulou $A\{y \mapsto c\}$. Ak sa vo formule X vyskytuje $\exists y A$ v oblasti platnosti všeobecných kvantifikátorov premenných $x_1, \dots, x_n$ ($X = \dots \forall x_1 (\dots \forall x_2 (\dots \forall x_n (\dots \exists y A \dots) \dots) \dots)$), tak pridáme do jazyka nový funkčný symbol, skolemovskú funkciu f a každý výskyt $\exists y A$ v X v oblasti platnosti kvantifikátorov $\forall x_1, \dots, \forall x_n$ nahradíme formulou $A\{y \mapsto f(x_1, x_2, \dots, x_n)\}$. Pre každú uzavretú formulu X v jazyku $\mathcal{L}$ existuje formula Y vo vhodnom rozšírení $\mathcal{L}'$ jazyka $\mathcal{L}$ taká, že Y neobsahuje existenčné kvantifikátory a X a Y sú ekvivalentné.

Existuje viacero algoritmov *konverzie formuly do klauzálnnej teórie*. My budeme používať nasledujúci algoritmus:

1. Implikácie nahradíme disjunkciami podľa ekvivalentných úprav 1.1.
2. Negačný normálny tvar (NNF): Presunieme negácie k atómom podľa ekvivalentných úprav 1.2 až 1.5.
3. Premenujeme premenné tak, aby každý kvantifikátor viazal inú premennú ako ostatné kvantifikátory podľa ekvivalentných úprav 1.18.

4. Skolemizácia: Existenčné kvantifikátory nahradíme substitúciou nimiviazaných premenných za skolemovské konštanty/aplikácie skolemovských funkcií na príslušné všeobecne kvantifikované premenné.
5. Prenexný normálny tvar (PNF): presunieme všeobecné kvantifikátory na začiatok formuly podľa ekvivalentných úprav 1.14 až 1.17.
6. Konjunktívny normálny tvar (CNF): distribuujeme disjunkcie do konjunkcií.
7. Odstránime konjunkcie rozdelením konjunktov do samostatne kvantifikovaných klauzúl.

Napríklad ekvisplniteľná klauzálna teória pre formulu

$$\exists x \forall y (\text{cat}(y) \rightarrow \text{loves}(x, y)) \wedge \forall x ((\text{cat}(x) \vee \text{loves}(x, \text{kitty})) \rightarrow \text{loves}(\text{kitty}, x))$$

v jazyku $\mathcal{L}$ s $\mathcal{C}_{\mathcal{L}} = \{\text{kitty}\}$, $\mathcal{F}_{\mathcal{L}} = \emptyset$, $\mathcal{P}_{\mathcal{L}} = \{\text{cat}^1, \text{loves}^2\}$ dostaneme následovne:

1. $\exists x \forall y (\neg \text{cat}(y) \vee \text{loves}(x, y)) \wedge \forall x (\neg (\text{cat}(x) \vee \text{loves}(x, \text{kitty})) \vee \text{loves}(\text{kitty}, x))$
2. $\exists x \forall y (\neg \text{cat}(y) \vee \text{loves}(x, y)) \wedge \forall x ((\neg \text{cat}(x) \wedge \neg \text{loves}(x, \text{kitty})) \vee \text{loves}(\text{kitty}, x))$
3. $\exists x \forall y (\neg \text{cat}(y) \vee \text{loves}(x, y)) \wedge \forall z ((\neg \text{cat}(z) \wedge \neg \text{loves}(z, \text{kitty})) \vee \text{loves}(\text{kitty}, z))$
4. $\forall y (\neg \text{cat}(y) \vee \text{loves}(a, y)) \wedge \forall z ((\neg \text{cat}(z) \wedge \neg \text{loves}(z, \text{kitty})) \vee \text{loves}(\text{kitty}, z))$
5. $\forall y \forall z ((\neg \text{cat}(y) \vee \text{loves}(a, y)) \wedge ((\neg \text{cat}(z) \wedge \neg \text{loves}(z, \text{kitty})) \vee \text{loves}(\text{kitty}, z)))$
6. $\forall y \forall z ((\neg \text{cat}(y) \vee \text{loves}(a, y)) \wedge ((\neg \text{cat}(z) \vee \text{loves}(\text{kitty}, z)) \wedge (\neg \text{loves}(z, \text{kitty}) \vee \text{loves}(\text{kitty}, z))))$
7. $T = \{\forall y \forall z (\neg \text{cat}(y) \vee \text{loves}(a, y)), \forall y \forall z (\neg \text{cat}(z) \vee \text{loves}(\text{kitty}, z)), \forall y \forall z (\neg \text{loves}(y, \text{kitty}) \vee \text{loves}(\text{kitty}, y))\}$

Rozšírený jazyk $\mathcal{L}'$ pre skolemizáciu má $\mathcal{C}_{\mathcal{L}'} = \{\text{kitty}, a\}$, $\mathcal{F}_{\mathcal{L}'} = \emptyset$, $\mathcal{P}_{\mathcal{L}'} = \{\text{cat}^1, \text{loves}^2\}$

Kapitola 2

Východiská práce

2.1 Prehľad existujúceho ekosystému

Pri výučbe predmetu Logika pre informatikov je používaná webová aplikácia Logic Workbook (elektronický pracovný zošit), ktorá bola vyvinutá ako súčasť predošlej bakalárskej práce [13]. Hlavným účelom Logic Workbooku je spájanie viacerých učebných pomôcok do jednej aplikácie. Učebné pomôcky pre zošit sú najprv vyvinuté ako samostatné aplikácie na základe knižnice React. Pre integrovanie aplikácie do zošita, je potrebné aplikáciu zverejniť ako knižnicu, ktorá exportuje funkciu `prepare` na inicializáciu aplikácie a Reactový funkcionálny komponent `AppComponent`. Do aplikácie sú integrované viaceré menšie aplikácie, ktoré tiež vznikli ako súčasti viacerých bakalárskych prác.

Jednou z integrovaných aplikácií je editor rezolvenčných dôkazov [11]. Editor ako vstup vyžaduje formuly v CNF tvare. Editor neobsahuje kontrolu skolemizácie a ekvivalentných úprav, čo je pri zadávaní komplexnejších cvičení problematické. Pre vyriešenie tohto problému vznikla táto bakalárska práca.

Ďalšou integrovanou aplikáciou je prieskumník štruktúr. Prieskumník štruktúr obsahuje dátový model na vyhodnocovanie výrazov, ktorý bol naposledy rozšírený ako súčasťou bakalárskej práce Reimplementácia prieskumníka štruktúr pre logiku prvého rádu [10]. Model je získaný nástrojom pre syntaktickú analýzu textových reťazcov vytvorený počas bakalárskej práce Prieskumník sémantiky logiky prvého rádu [9]. V modeli a nástroji pre syntaktickú analýzu ale chýba reprezentácia tautológie a nespĺniteľnej formuly, ktoré potrebujeme pre kontrolu viacerých ekvivalentných úprav (napríklad úpravy 1.9), preto počas implementácie tejto bakalárskej práce ich rozšírime.

2.2 Použité technológie

V tejto podkapitole zhrnieme a krátko opíšeme technológie, ktoré sme použili pri vývoji aplikácie. Technológie boli zvolené na základe toho, že sú použité v iných aplikáciách integrovaných do Logic Workbook alebo v samotnom pracovnom zošite.

2.2.1 HTML, CSS, Bootstrap

HTML [3], alebo HyperText Markup Language, je základný jazyk, ktorý sa používa na tvorbu webových stránok. Primárne využitie HTML je na definovanie štruktúry webovej stránky. HTML dokumenty sú tvorené textom a elementami v stromovej štruktúre. Elementy dokumentu sa označujú začiatočnými a koncovými značkami a môžu mať atribúty. Atribúty prvku riadia ako sa správajú. Na zmenenie vizuálu webových stránok sa používa CSS [2], alebo Cascading Style Sheets. CSS je jazyk, ktorý popisuje ako sa elementy HTML zobrazujú v rôznych médiách. Bootstrap [1] je populárny front-end toolkit, ktorý poskytuje predpripravené HTML komponenty, ako sú napríklad tlačidlá, tooltipy atď. Bootstrap tiež poskytuje CSS štýly.

2.2.2 TypeScript

JavaScript je programovací jazyk, ktorý umožňuje dynamicky meniť obsah webových stránok. TypeScript [7] je jazyk založený na JavaScripte. TypeScript obsahuje silné typovanie, čo pomáha pri vývoji ľahším odhalením chýb. Pred spustením aplikácie sa TypeScript prekladá do JavaScriptu, čo umožňuje kompatibilitu s JavaScriptom.

2.2.3 React, React-Bootstrap

React [4] je knižnica pre JavaScript, ktorá slúži na vytvorenie používateľského rozhrania webových aplikácií. Základným stavebným kameňom Reactu sú komponenty. Každý komponent je definovaný ako JavaScriptová funkcia, ktorá reprezentuje časť rozhrania. Komponenty sú definované v súboroch s príponou „.jsx“, v ktorých je možné kombinovať JavaScript s HTML, pričom volanie komponentov sa tiež zapisuje ako HTML elementy. Výstup komponentu je JSX element, ktorý React synchronizuje s HTML, ktoré zobrazuje webový prehliadač. Komponenty môžu mať stav, ktorý React kontroluje. Ak sa stav komponentu zmení, React aktualizuje iba príslušnú časť rozhrania. Na využitie stavu React používa takzvané *hooky*. React-Bootstrap [5] je knižnica, ktorá prepája React s Bootstrapom. Umožňuje vytvárať Bootstrapove komponenty v tvare Reactových komponentov.

2.2.4 Redux

Redux [6] je knižnica v jazyku JavaScript určená na spravovanie globálneho stavu v aplikácii. Redux používa nemenné úložisko (store) na ukladanie globálneho stavu. Stav uložený v store je jednoduchý (plain) JavaScriptový objekt, ktorý je možné meniť iba ako celok. Na zmenu stavu používajú *reducery* a *akcie*. Reducer je funkcia, ktorá na vstupe dostane stav a akciu. Reducer vráti kópiu stavu zmenenú podľa akcie, ktorú uloží do store. Akcia je objekt, ktorý popisuje identifikátor zmeny stavu, a prípadne obsahuje ďalšie voliteľné parametre. Na získanie aktuálneho stavu Redux používa *selektory*. Selektor je funkcia, ktorá na vstupe dostane stav a vráti informáciu získanú zo stavu. Redux sa často používa s Reactom na manažovanie stavu Reactových komponentov. Reactové komponent môžu používať hooky pre použité selektorov a zmenu stavu.

Kapitola 3

Požiadavky a návrh

V tejto kapitole zhrnieme požiadavky aplikácie a opíšeme jej návrh. V návrhu opíšeme internú štruktúru aplikácie a na vyššej úrovni opíšeme kontrolu ekvivalentných úprav.

3.1 Požiadavky

Požiadavky vyplývajú z toho, že celkovým cieľom aplikácie je kontrola ekvivalentných úprav. Prvou požiadavkou aplikácie je, že používateľ musí byť schopný pridávať osobitné formuly. Aplikácia musí byť schopná kontrolovať či formuly sú správne napísané, prípadne oznámiť používateľovi ak urobil chybu. Aby bola možná kontrola správneho napísanie formuly aplikácia potrebuje jazyk, v ktorom sú formuly napísané. Preto druhou požiadavkou je aby bolo možné zadať aplikácii mimologické symboly jazyka formúl.

Hlavnou požiadavkou je samotná kontrola úprav. Kontrolovanie viacerých rôznych úprav by mohlo viesť k tomu že študent nepochopí úplne tomu čo robí, preto aplikácia bude kontrolovať iba jeden typ úpravy v jednom kroku. Používateľ vyberie akú úpravu má pre dva dané formuly kontrolovať. Počas vývoja sa ale ukázalo, že aby bol kontrolór pohodlnejší na používanie je potrebné kontrolovať aj sofistikovanejšie úpravy, ako je napríklad viacnásobná asociativita a komutativita. Hlavné použitie aplikácie je pre kontrolu prevodu formuly do CNF tvaru. Aplikácia by preto mala pracovať s postupnosťami ekvivalentných úprav a mala byť schopná pracovať s viacerými postupnosťami.

Aplikácia je určená pre výučbu predmetu Logika pre informatikov, kde sa používa elektronický pracovný zošit, vyrobený ako súčasť inej bakalárskej práce [13]. Väčšina úloh pre študentov predmetu je zadávaná a riešená v elektronickom zošite, preto požiadavkou je, že aplikácia by mala byť do neho integrovaná. Táto požiadavka znamená, že používateľské rozhranie bude vyrobené pomocou knižnice React a tiež že bude možné stav aplikácie ľahko uložiť a načítať.

3.2 Návrh

3.2.1 Návrh stavu aplikácie

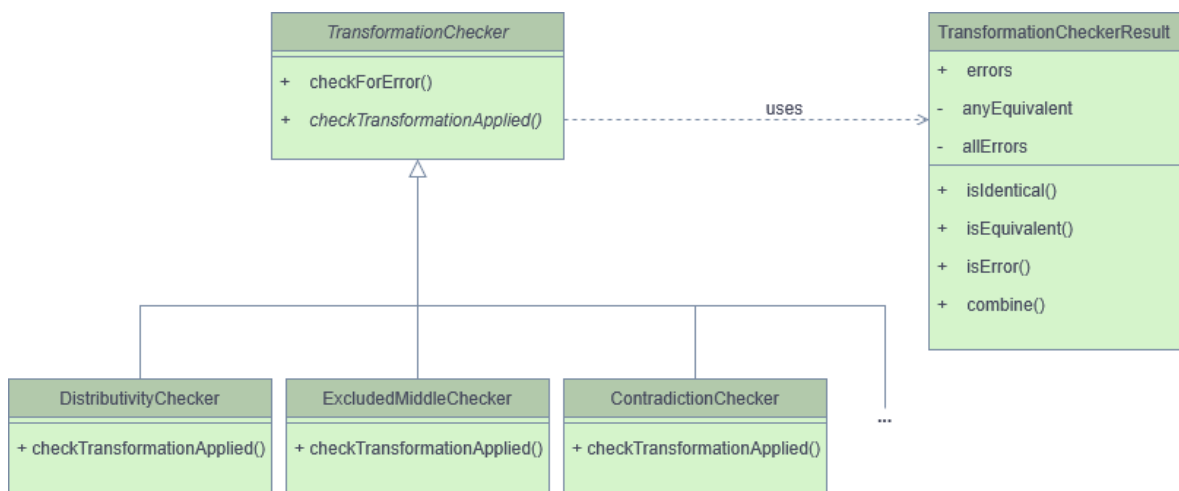
Stav aplikácie bude rozdelený na 3 časti: jazyk, úpravy a ukladanie/načítanie. Každá časť bude uchovávať iba nevyhnutné dáta. V stave nebude potrebné ukladať komplexnejšie dáta, ktoré vieme odvodiť z uložených.

Prvá časť stavu bude určená pre logický jazyk, konkrétne pre mimologické symboly. Jazyková časť si bude pamätať text vložený do textových polí pre konštanty, predikáty a funkcie. Všetky formuly využívajú logický jazyk pri premene do stromových reprezentácií. Kvôli efektívnosti je ideálne vyhnúť sa použitiu chybným vstupom mimologických symbolov, preto si jazyková časť bude tiež pamätať posledné validné vstupy pre texty, ktoré sa použijú pri kontrolovaní formúl.

Druhá časť stavu bude pre samotné formuly a úpravy. Časť si bude pre každú formulu pamätať jej jedinečné identifikačné číslo, text z textového polia, zvolenú ekvivalentnú úpravu a číslo predošlej formuly (číslo formuly, na ktorú sa úprava aplikovala). Keďže hlavným účelom aplikácie je prevod formuly do CNF tvaru postupným aplikovaním ekvivalentných úprav, v stave budú tiež zapamätané postupnosti identifikačných čísel formúl a tiež identifikačné číslo danej postupnosti. Pre podporu skolemizácie bude v tejto časti vstupy textových polí pre skolémové konštanty a funkcie a identifikačné číslo formuly, pri ktorej vznikli.

Posledná časť bude určená pre ukladanie a načítanie stavu. Jediné čo bude v tejto časti zapísané je chybová hláška ak načítanie stavu zlyhá.

3.2.2 Návrh kontroly úprav



Obr. 3.1: Diagram tried kontroly ekvivalentných úprav

Hlavný problém pri kontrole ekvivalentných úprav je, že úprava mohla nastať ľu-

bovolný počet krát v ľubovoľnej časti formuly. Ak by sme kontrolovali každé miesto kde by mohla nastať úprava, riešenie by bolo príliš neefektívne a pomalé. Ale môžeme využiť to, že jedna formula vznikla úpravou druhej formuly, a preto buď je časť prvej formuly rovnaká s časťou druhej formuly, alebo nastala úprava. Ak pri súčasnom rekurzívnom prechádzaní stromových reprezentácií obidvoch formúl nájdeme rozdiel, môžeme usúdiť že sa používateľ na danom mieste pokúsil urobiť ekvivalentnú úpravu. Problém nastane keď niektoré ekvivalentné úpravy nemenia časti formúl, ako napríklad formuly 1.8, kde obidve formuly zdieľajú tú istú spojku. Týmto spôsobom by sme preskočili rovnakú spojku a našli rozdielne neekvivalentné podformuly, z toho by sme zle usúdili, že nastala chyba. Toto vyriešime tým, že chyby budú „prebublávať“ hore. Ak sa program rekurzívne dostal do koreňa A a našiel chybu v jeho podstrome tak sa pozrie, či nenastala úprava v koreni A . Riešenie sa síce spomalí, ale vďaka tomu že kontrolór je určený ako učebná pomôcka, stromové reprezentácie vstupných formúl nebudú príliš hlboké, a preto spomalenie je zanedbateľné.

Výsledok kontroly bude reprezentovaný osobitnou triedou. Trieda výsledku bude obsahovať metódy na zistenie výsledku a tiež metódu na kombinovanie výsledkov. Možné hodnoty výsledku budú tri: identický, ekvivalentný a chybný. Identický výsledok znamená, že podstromy sú identické. Ekvivalentný výsledok je vrátený, keď v podstrome nastala aspoň jedna ekvivalentná úprava a všetky úpravy sú bezchybné. Chybný výsledok znamená, že v podstrome nastala buď chybná úprava, alebo iná chyba. Navyše obsahuje chybnú hlášku, ktorá sa neskôr zobrazí používateľovi.

Rôzne ekvivalentné úpravy budeme reprezentovať osobitnými triedami pre každú úpravu. Triedy budú odvodené z abstraktnej triedy, ktorá bude obsahovať spoločnú časť algoritmu kontroly. Triedy sa budú líšiť v časti, ktorá usúdi či na konkrétnom mieste formuly nastala ekvivalentná úprava. Diagram tried je zobrazený v obrázku 3.1. Pseudokód spoločnej rekurzívnej časti kontroly je nasledovný:

```

checkForError(orig, trans) {
    if (sameFunctor(orig, trans)) {
        result = checkChildren(orig, trans)
        if (result.isEquivalentOrIdentical()) {
            return result
        }
    }
    return checkTransformationApplied(orig, trans)
}

```

Metódy `sameFunctor` a `checkChildren` budú súčasťou abstraktnej triedy. Funkcia `sameFunctor` zistí či korene stromov sú rovnaké (či sú obidva korene rovnaké spojky, rovnaké kvantifikátory s rovnakými premennými atď.). Keďže stromové reprezentácie

formúl sú n-árne metóda `checkChildren` zavolá rekurzívnu kontrolu koreňov všetkých podstromov a skombinuje ich výsledky. Metóda `checkTransformationApplied` je abstraktná, ktoré konkrétne triedy prepíšu. Ak ekvivalentná úprava nastala, tak bude potrebné zavolať rekurziu na iné podstromy než tie ktoré skontrolovala `checkChildren`. Z toho dôvodu metóda tiež zavolá rekurzívnu kontrolu všetkých potrebných podstromov.

Kapitola 4

Implementácia

V tejto kapitole bližšie opíšeme konkrétnu implementáciu aplikácie. Začneme krátkym opisom súborovej štruktúry projektu, potom popíšeme ako a prečo sme rozšírili model formúl a nástroj na syntaktickú analýzu. Za tým opíšeme implementáciu kontroly ekvivalentných úprav. Viacero úprav sa kontroluje podobným spôsobom, preto opíšeme ako sa kontroluje jedna z ľahších úprav a tiež opíšeme niektoré úpravy, ktoré majú zložitejšiu kontrolu. Nakoniec opíšeme používateľské rozhranie a implementáciu kompatibility pre logický pracovný zošit.

4.1 Súborová štruktúra

Všetky zdrojové kódy aplikáciu sú v priečinku `src`. Štruktúra priečinku je nasledovná:

- `error_checkers` - obsahuje súbory tried jednotlivých kontrolórov úprav. Taktiež obsahuje abstraktnú triedu, z ktorej sú zdedené kontrolóry
- `features` - v osobitných priečinkoch obsahuje jednotlivé časti stavu spomenuté v návrhu (3.2.1). Tiež obsahuje Reactové komponenty k príslušným častiam
- `model` - obsahuje rozšírený model formúl
- `state` - obsahuje Reduxový store a základné Reactové hooky pre jeho použitie
- `tests` - jednotkové testy pre jednotlivé triedy kontroly ekvivalentných formúl

Samotný priečinok `src` obsahuje súbory:

- `App.css` a `App.tsx` - CSS štýly a Reactový komponent celej aplikácie
- `LogicContext.ts` a `AppComponent.tsx` - pre integráciu aplikácie do Logic Workbooku
- `index.ts` a `main.tsx` - použité pri vývoji aplikácie príkazom `npm run dev`

4.2 Rozšírenie modelu formúl a nástroja na syntaktickú analýzu

Pre kontrolu ekvivalentných úprav potrebujeme porovnávať rôzne formuly. Rozhodli sme sa použiť stromový dátový model pre formuly vytvorený počas predošlej bakalárskej práce Jozefa Filipa [10] pre reprezentáciu formúl. Taktiež na získanie modelu z textového reťazca použijeme nástroj na syntaktickú analýzu vytvorený počas bakalárskej práce Milana Cifry [9]. Tento nástroj budeme ďalej nazývať parser. Model je výhodný v tom, že nám umožňuje ľahko prechádzať formulou. Problémom je, že viaceré ekvivalentné úpravy pracujú s tautológiami a nesplniteľnými formulami (napríklad úpravy (1.9)). Preto bolo potrebné rozšíriť model pridaním tried, ktoré ich budú reprezentovať. Aby parser používal naše triedy a vedel rozpoznať textové znaky, ktoré im zodpovedajú, sem ho museli rozšíriť. Ďalším problémom je, že pre kontrolu skolemizácie sú potrebné nové skolemove konštanty a funkcie. Pre jednoduchosť používateľského rozhrania by bolo ideálne použiť jeden textový vstup pre ich zadávanie. Parser má síce možnosť analyzovať konštanty alebo funkcie z textu, ale nemá možnosť ich analyzovať naraz z jedného textu. Parser sme teda rozšírili aj o túto funkcionálnosť.

Model reprezentuje jednotlivé časti formúl (ako napríklad konjunkcia, implikácia, negácia atď.) rôznymi triedami. Tieto triedy sú odvodené od abstraktnej triedy **Formula**. Trieda **Formula** a abstraktná trieda **Term**, z ktorej sú odvodené triedy pre reprezentáciu termov, sú odvodené z abstraktnej triedy **Expression**. Pre reprezentáciu tautológie a nesplniteľnej formuly sme pridali triedy **AlwaysTrue** a **AlwaysFalse** ako potomkov triedy **Formula**. Ukážka triedy **AlwaysTrue** je v obrázku (4.1) a trieda **AlwaysFalse** je jej podobná. Obidve triedy majú konštruktor bez parametrov a metódu, o ktorej povieme viac nižšie. Parser bol vytvorený pomocou knižnice **pegjs** a pre syntaktickú analýzu používa súbor **grammar.pegjs** v ktorom je zadaný syntax, podľa ktorého sa text analyzuje. Pre vytvorenie konkrétnej štruktúry parser používa takzvané „factories“. Pre rozšírenie analýzy sme pridali dve nové factories a rozšírili súbor **grammar.pegjs**. Taktiež sme do súboru pridali časti potrebné pre analýzu skolemovských konštánt a funkcií.

Jednou zo zložitejších úprav, ktoré v našej aplikácii kontrolujeme je viacnásobná asociativita a komutativita (úpravy (1.7) a (1.8)). Úprava umožňuje usporiadať podformuly spojené konjunkciami alebo podformuly spojené disjunkciami do ľubovoľného usporiadania. Kontrolovať túto úpravu pomocou stromového modelu a algoritmu popísaného v návrhu (3.2.2) by bolo veľmi ťažké. Pri každej viacnásobnej konjunkcii alebo disjunkcii potrebuje skontrolovať či majú tie isté podformuly. Zároveň štruktúru stromu ovplyvňuje asociativita. Asociativitu vyriešime tým, že strom sploštíme nahradením viacerých binárnych uzlov konjunkcie a disjunkcie jedným n -árnym uzlom. Komutati-

Ukážka kódu 4.1: Trieda reprezentujúca tautológiu

```

class AlwaysTrue extends Formula {
    constructor() {
        super([], "", "");
    }

    toString(): string {
        return '⊤';
    }

    flatten() {
        return new AlwaysTrue();
    }
}

```

vitú vyriešime tým, že podformuly zoradíme podľa vybraného kanonického usporiadania. Ak sú dve formuly navzájom ekvivalentné, tak po sploštení a usporiadaní budú identické. Toto sploštenie a usporiadanie sme sa rozhodli implementovať rozšírením modelu pridaním novej metódy **flatten**.

Metódu **flatten** sme definovali ako abstraktnú metódu v abstraktnej triede **Expression**. Metóda **flatten** vytvorí a vráti kópiu daného stromu, pričom kópia je sploštená a usporiadaná. Metóda najprv vytvorí kópie podstromov zavolaním **flatten** na podstromy, a potom ich použije ako parametre pre vytvorenie nového objektu rovnakej triedy ako tej, na ktorej bola zavolaná. Samotné sploštenie a usporiadanie je implementované v triedach pre konjunkciu a disjunkciu. Algoritmus pre disjunkciu je podobný algoritmu pre konjunkciu, preto opíšeme iba algoritmus pre konjunkciu. Pri konjunkcii, po vyrobení kópie uzla metóda pre obidva podstromy porovná či trieda koreňa podstromu je tiež konjunkcie. Ak áno, tak spojí pole podstromov koreňa s poľom podstromov koreňa podstromu. Keďže podstrom vznikol metódou **flatten** je zaručené, že ani jeden podstrom konjunkcie nie je konjunkcia, preto je strom sploštený. Po sploštení metóda usporiada pole podstromov. Pre usporiadanie sme implementovali metódu **compare** tiež ako abstraktnú metódu v triede **Expression**. Metóda **compare** rozhodne ako usporiadať stromy, tým že najprv porovná mená konštruktorov, potom porovná prvky, ktoré majú niektoré triedy (ako napríklad **name** pri triedach termov alebo **variableName** pri triedach kvantifikátorov) a nakoniec zaradom porovná podstromy.

Ukážka kódu 4.2: Kontrola tried dátového modelu

```

checkRequisites(original: Expression, transformed: Expression) {
    return (original instanceof Implication &&
            transformed instanceof Disjunction &&
            transformed.subLeft instanceof Negation);
}

```

4.3 Kontrola ekvivalentných úprav

Ako bolo spomenuté v návrhu (3.2.1) každá ekvivalentná úprava bude mať osobitnú triedu pre kontrolu jej správnosti. Každá trieda bude obsahovať metódu `checkTransformationApplied`, ktorá rozhodne či ekvivalentná úprava nastala a zavolá hlavný algoritmus na relevantné podstromy. Keďže viacero tried implementuje túto metódu veľmi podobne, ukážeme si iba jednu z nich. Tou triedou je trieda `ImplicationEliminationChecker`. Potom si ukážeme pár tried, ktorých implementácie je trochu zložitejšia.

4.3.1 ImplicationEliminationChecker

`ImplicationEliminationChecker` je trieda, ktorá kontroluje ekvivalentnú úpravu eliminácie implikácie (ľavá úprava (1.1)). Metóda `checkTransformationApplied` má dva povinné parametre, ktorými sú formuly v stromovej reprezentácii a jeden nepovinný parameter pre výsledok kontroly podstromov, ktoré metóda dostane pri „prebublávaní“ chyby nahor popísanej v návrhu. Metóda najprv skontroluje to, či úprava nastala tým, že skontroluje triedy v stromovej reprezentácii dvojice formúl. Na to používa pomocnú metódu `checkRequisites`, ktorá je zobrazená na obrázku (4.2). Keďže úprava mohla byť aplikovaná aj v opačnom smere, ak metóda `checkRequisites` vráti `false`, tak sa znovu zavolá s vymenenými parametrami. Ak aj vtedy `checkRequisites` vráti `false`, tak metóda `checkTransformationApplied` vie, že úprava nenastala správne. Potom sa pozrie či dostala výsledok kontroly podstromov, ak áno tak sa rozhodne, či vráti svoju chybovú hlášku, alebo či vráti výsledok kontroly podstromov. V opačnom prípade, tak vráti výsledok s chybovou hlášku. Ak aspoň raz metóda `checkRequisites` vrátila výsledok `true`, tak úprava nastala správne a metóda `checkTransformationApplied` zavolá hlavný algoritmus `textttcheckForError` z návrhu pre správne podstromy. Konkrétne pre ľavý podstrom implikácie s podstromom negácie a pre pravý podstrom implikácie a pravý podstrom disjunkcie. `checkTransformationApplied` potom výsledky skombinuje, a keď je výsledok identický alebo ekvivalentný tak metóda vráti ekvivalentný výsledok. Ak výsledok je chybný, vráti ho, pretože obsahuje relevantnejšiu chybovú hlášku.

Ukážka kódu 4.3: Interface skolemizedPattern

```
interface skolemizedPattern {
    name: string ,
    subterms: string []
}
```

4.3.2 ReorderingChecker

Trieda **ReorderingChecker** kontroluje viacnásobnú asociativitu a komutativitu. Používa pri tom metódu dátového modelu **flatten**, ktorú sme mu pridali. Metóda je popísaná v predošlej podkapitole (4.2). Kontrolovanie správnosti pomocou tejto metódy je potom jednoduché. Po použití metódy sa stráca informácia o tom, či sú formuly navzájom identické. Preto trieda najprv skontroluje identickosť formúl, tým že zavolá hlavný algoritmus **textttcheckForError** na formuly, ktoré jej boli dané na vstupe. Ak sú identické, tak ihneď vráti identický výsledok. Ináč zavolá hlavný algoritmus **textttcheckForError** na sploštených a usporiadaných formulách vytvorených pomocou metódy **flatten**. Ak výsledok je teraz identický znamená to, že nastala správna úprava, a preto trieda vráti ekvivalentný výsledok. Ináč vráti chybný výsledok s chybovou hláškou.

4.3.3 SkolemizationChecker

Trieda **SkolemizationChecker** kontroluje správnosť skolemizácie. Skolemizácia z formuly vymaže existenčné kvantifikátory a premenné, ktoré kvantifikátory viazali nahradí novými skolemovými funkciami a konštantami. Ak sa existenčný kvantifikátor nachádzal v oblasti platnosti niektorých všeobecných kvantifikátorov, nahradí premennú novou skolemovou funkciou, ktorej podtermy sú všetky premenné viazané tými všeobecnými kvantifikátormi. Ináč nahradí premennú skolemovou konštantou. Pre kontrolu skolemizácie trieda **SkolemizationChecker** bude používať viacero súkromných atribút. Tie možno vidieť v obrázku (4.4). Pred začiatkom kontroly budú triede priradené nové skolemové konštanty a funkcie, ktoré používateľ zadal. Tie trieda uchová v poliach v objekte **allowedSkolemSymbols**. Trieda bude súčasne rekurzívne prechádzať dvojicu formúl v stromovej reprezentácii. Ak narazí na všeobecný kvantifikátor, pridá meno premennej, ktorú kvantifikátor viaže na koniec poľa **universalQuants**. Ak narazí na existenčný kvantifikátor v prvej formule, ale nie v druhej formule, tak usúdi, že tento existenčný kvantifikátor bude skolemizovaný. Zapiše si do mapy **changedVars** nový objekt typu **skolemizedPattern**, kde **name** je prázdny reťazec, a **subterms** je kópia poľa **universalQuants**. Kľúčom je meno premennej, ktorú existenčný kvantifikátor

Ukážka kódu 4.4: Atribúty triedy SkolemizationChecker

```

universalQuants: string []
changedVars: Map<string, skolemizedPattern>
allowedSkolemSymbols: {constants: string [], functions: SymbolWithArity}
usedSymbols: Set<string>

```

viazal. Interface typu `skolemizedPattern` je v obrázku (4.3). `name` reprezentuje meno funkcie alebo konštanty, ktorá nahradí premennú a `subterms` reprezentuje premenné viazané všeobecnými kvantifikátormi. `name` je zatiaľ prázdne, lebo trieda nevie aká konštantá alebo funkcia bude nahrádzať danú premennú. Keď trieda narazí na premennú, skontroluje či premenná má byť skolemizovaná, tým že pozrie či mapa `changedVars` obsahuje meno premennej ako kľúč. Ak neobsahuje, tak trieda ďalej pokračuje v algoritme. Ináč skontroluje či druhá formula obsahuje správnu skolemovú konštantu alebo funkciu. Kontrolu urobí tak, že sa zoberie objekt typu `skolemizedPattern` z mapy. Ak dĺžka poľa `subterms` je nulová, tak koreň druhej formuly musí byť konštantá. Ak je nenulová, tak musí byť funkcia. Ďalej sa pozrie na `name`. Ak `name` je prázdny reťazec, tak skontroluje meno konštanty alebo funkcie. Ak `name` je prázdny reťazec, tak trieda usúdi že skolemovu konštantu alebo funkciu vidí prvýkrát. Pozrie či je meno v `allowedSkolemSymbols` a či nebolo už použité pre skolemizáciu iného kvantifikátor tým že pozrie do množiny `usedSymbols`. Ak meno je nová skolemova funkcia alebo konštantá, ktorá nebola použitá, tak `name` prepíše a pridá ho do množiny `usedSymbols`. Navyše pre skolemové funkcie, pozrie či funkcia obsahuje všetky premenne, ktoré sú zapísané v `subterms`.

4.4 Implementácia používateľského rozhrania

Používateľské rozhranie je vytvorené pomocou knižnice React. Rozhranie sa skladá z Reactových komponentov. Aplikácia sa skladá z komponentu `App`, ktorý obsahuje tri komponenty. Prvý komponent `ImportExportComponent` obsahuje tlačidlá pre uloženie a načítanie stavu. Druhý komponent `LanguageComponent` slúži pre zadávanie jazyka a skladá sa z troch komponentov `LanguageInput`, ktoré slúžia na zadávanie mimologických symbolov. Tretí komponent `TransformationsComponent` slúži ako hlavná časť aplikácie. Pre každú postupnosť zo stavu vytvorí komponent `SequenceComponent`. `SequenceComponent` následne vytvorí `FormulaComponent` pre každú formulu z postupnosti úprav. Každá formula postupnosti okrem prvej má tlačidlo pre vyber úpravy, ktorá sa kontroluje. Výhodou použitia knižnice React je, že komponenty sa prekresľujú iba ak je to potrebné. Napríklad ak sa zmení text formuly, prekreslí sa iba

The screenshot displays the Logic Workbook interface. At the top right are 'Import' and 'Export' buttons. The 'Language $\mathcal{L}$ ' section includes input fields for 'Individual constants' (containing $\mathcal{C}_{\mathcal{L}} = \{ a \}$), 'Predicate symbols' (containing $\mathcal{P}_{\mathcal{L}} = \{ A/1, B/1, C/2 \}$), and 'Function symbols' (containing $\mathcal{F}_{\mathcal{L}} = \{ f/1 \}$). Below this, 'Transformation Sequence 1' shows a series of logical steps: starting with $(\forall x (\neg A(f(x)) \wedge B(x)) \rightarrow \forall z C(a,z))$, it applies 'Implication Elimination' to get $(\neg \forall x (\neg A(f(x)) \wedge B(x)) \vee \forall z C(a,z))$, then 'DeMorgan' to get $(\exists x (A(f(x)) \vee \neg B(x)) \vee \forall z C(a,z))$, and finally 'Skolemization' to reach the goal b . 'Transformation Sequence 2' starts with $(\forall x \exists y C(x,y) \vee \forall x \neg (B(x) \rightarrow A(f(x))))$ and shows an attempt at 'Implication Elimination' which results in a red error message: 'Formula is identical to previous formula!'. A '+ Add Transformation Sequence' button is at the bottom.

Obr. 4.1: Ukážka používateľského rozhrania

FormulaComponent a LanguageComponent sa neprekreslí. Ukážka rozhrania je v obrázku (4.1).

4.5 Implementácia knižnice pre Logic Workbook

Pre integráciu aplikácie do logického pracovného zošita je potrebná funkcia `prepare` a Reactový komponent `AppComponent`. Funkcia `prepare` slúži pre inicializáciu aplikácie. Funkcia má jeden nepovinný vstupný parameter `initialState`, podľa ktorého sa inicializuje stav aplikácie. Výstup funkcie je objekt `instance` a funkcia `getState`. Objekt `instance` obsahuje stav aplikácie. Funkcia `getState` vráti stav aplikácie vo forme JSON súboru. Komponent `AppComponent` sa bude zobrazovať v logického pracovnom zošite. V rámci komponentu používame náš komponent `App`. Komponent `App` je obalený Reduxovým komponentom `Provider`, ktorý umožní aplikácii prístup k stavu.

Kapitola 5

Testovanie

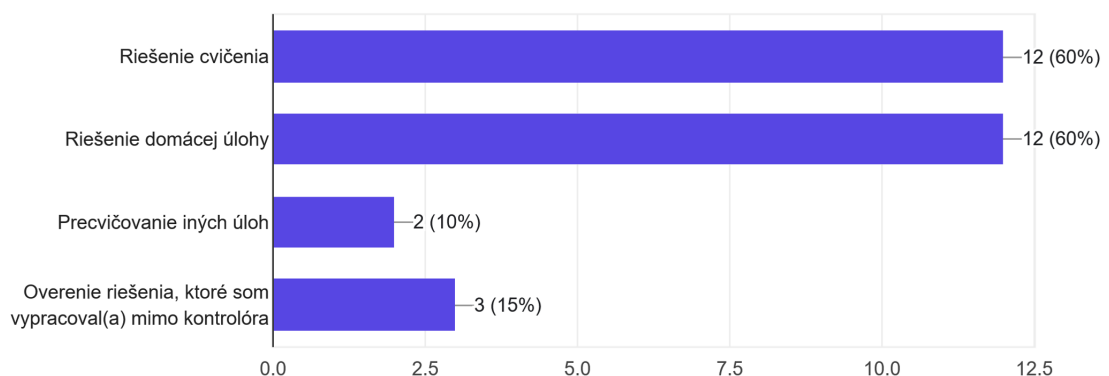
V tejto kapitole zhrnieme ako bola aplikácia testovaná a aké výsledky vyplynuli z testovania.

5.1 Spôsob testovania

Aplikácia bola testovaná počas letného semestra v rámci predmetu Logika pre informatikov ako súčasť cvičení a domácej úlohy. Súčasťou domácej úlohy pre študentov bola úloha na úpravu formúl do prvorádovej CNF. Zadanie úlohy je v prílohe A (5.2). Keďže aplikácia nebola v ten čas integrovaná do elektronického pracovného zošita, aplikácia bola študentom zverejnená cez stránku GitHub. Nasledovné študenti mali možnosť vyplniť dotazník o ich skúsenostiach s aplikáciou. Kópia dotazníka sa nachádza v prílohe B (5.2).

Na aký účel ste kontrolór použili?

20 responses



Obr. 5.1: Na aký účel ste kontrolór použili? Výsledky sú v počte študentov

Dotazník sa skladal z pätnástich otázok, z ktorých prvé tri sa zaoberali spôsobom, akým študenti použili aplikáciu. Potom nasledovalo desať otázok podľa štandardu SUS [8]. Dotazník štandardu SUS obsahuje desať otázok na ktoré, je možné odpovedať piatimi odpoveďami od „úplne nesúhlasím“ po „úplne súhlasím“. Každá odpoveď je ohodnotená číslom 1 až 5, kde úplne súhlasím je za 5 bodov. Z odpovedí sa vypočíta SUS skóre, ktoré zodpovedá použiteľnosti aplikácie. Čím vyššie je SUS skóre, tým je aplikácia použiteľnejšia. Posledné dve otázky boli otvorené, študenti v nich mohli vyjadriť konkrétne čo sa im páčilo, respektíve nepáčilo na aplikácii.

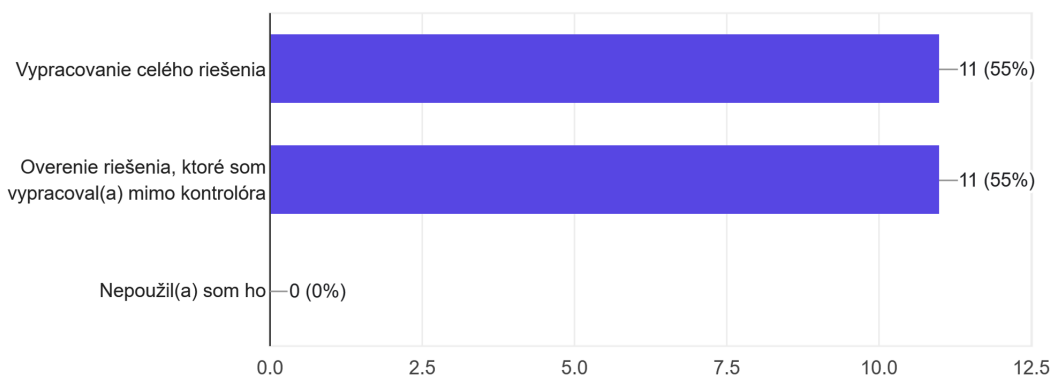
5.2 Výsledky testovania

Dotazník vyplnilo 20 študentov, pričom všetci aplikáciu použili. Na grafe v obrázku 5.1 vidno, že najčastejšie použitie aplikácie, bolo pre riešenie zadaných úloh. Pár študentov tiež použili aplikáciu pre precvičovanie iných úloh. Graf na obrázku 5.2 ukazuje, že iba približne polovica študentov použila aplikáciu pre vypracovanie celého riešenia. Veľká časť respondentov ju použila pre overenie ich riešenia. Je pravdepodobné, že ak by bola aplikácia počas testovania integrovaná do Logic Workbooku, tak by časť študentov, ktorý aplikáciu použili pre celé riešenie vzrástla.

Pri počítaní SUS skóre, každá nepárna otázka je sformulovaná pozitívne a prispieva svojou hodnotou k skóre. Naopak každá párna otázka je sformulovaná negatívne a znižuje skóre. Vypočítané priemerné SUS skóre vyšlo 72.875 čo znamená, že použiteľnosť aplikácie je dobrá. Najlepšie skóre mala otázka „Vedel(a) som si predstaviť, že väčšina ľudí sa naučí používať kontrolór veľmi rýchlo.“. Otázka mala priemernú hodnotu 4.15 čo znamená, že v priemere študenti súhlasili s touto otázkou. V otvorenej časti formulára

Akým spôsobom ste kontrolór použili?

20 responses



Obr. 5.2: Akým spôsobom ste kontrolór použili? Výsledky sú v počte študentov

viacerý študenti napísali, že sa im na aplikácii páčila jej jednoduchosť. Pár študentov ocenili, že aplikácia kontroluje samostatné ekvivalentné úpravy, a že používať aplikáciu bolo lepšie ako meniť formuly ručne.

Najhoršie skóre mala otázka „Potreboval(a) som sa naučiť mnoho vecí predtým, ako som mohol(a) začať pracovať s kontrolórom.“. Otázka mala priemernú hodnotou 2.2, čiže študenti v priemere nesúhlasili s tvrdením. Najčastejšou pripomienkou bolo to, že chybové hlášky neboli dosť nápomocné. Táto pripomienka sa dá riešiť iba do určitej miery, lebo s konkrétnejšími chybovými hláškami rastie šanca, že aplikácia zle určí kde vznikla chyba, čo môže študentov zmiast.

Záver

V rámci tejto práce sme vyvinuli webovú aplikáciu určenú pre kontrolu ekvivalentných úprav. Vyvíjali sme ju iteratívno-inkrementálnym prístupom. Najprv sme aplikáciu implementovali s minimálnou funkcionalitou a postupne ju vyvíjali pridávaním ďalších funkcionalít. Navrhli sme stav aplikácie a implementovali ho pomocou knižnice Redux. Postupne sme pridávali do aplikácie kontrolu rôznych ekvivalentných úprav a skolemizácie. Potrebovali sme pri tom rozšíriť dátový model pre reprezentáciu formúl vytvorený počas bakalárskej práce Jozefa Filipa. Taktiež sme rozšírili nástroj na syntaktickú analýzu vytvorený počas bakalárskej práce Milana Cifry.

Nakoniec sme aplikáciu otestovali počas výučby predmetu Logika pre informatikov. Študenti po práci s aplikáciu vyplnili dotazník, podľa ktorého sme zhodnotili, že študenti ocenili jednoduchosť aplikácie. Tiež sme zhodnotili, že použiteľnosť aplikácie je dobrá. Výsledná verzia aplikácie je vhodná na použitie pri výučbe predmetu.

Existuje stále priestor na zlepšenie v viacerých oblastiach. Najčastejšou pripomienkou študentov boli chybové hlášky pri nesprávnej úprave. Jedným zlepšením by preto mohlo byť ich vylepšenie, poprípade skonkretizovanie. Ďalším možným zlepšením by mohla byť samotná kontrola úprav. V tejto chvíli aplikácia vie kontrolovať iba jeden typ úpravy. Vylepšením by mohlo byť to, že niektoré základné úpravy (ako napríklad asociativita a komutativita) by sa kontrolovali aj pri iných úpravách.

Vďaka tejto práci, sme mali možnosť zoznámiť sa s tvorbou moderných webových aplikácií. Práca nás naučila o knižniciach React a Redux, ktoré boli pre nás nové. Zoznámili sme sa so správou stavu webových aplikácií. Tiež sme nadobudli cenné skúsenosti s vývojom aplikácie a práce správou verzií kódu pomocou platformy GitHub.

Literatúra

- [1] Online dokumentácia bootstrap. <https://getbootstrap.com/>.
- [2] Online dokumentácia CSS. <https://developer.mozilla.org/en-US/docs/Web/CSS>.
- [3] Online dokumentácia HTML. <https://developer.mozilla.org/en-US/docs/Web/HTML>.
- [4] Online dokumentácia react. <https://react.dev/>.
- [5] Online dokumentácia react-bootstrap. <https://react-bootstrap.netlify.app/>.
- [6] Online dokumentácia redux. <https://redux.js.org/>.
- [7] Online dokumentácia typescript. <https://www.typescriptlang.org/>.
- [8] John Brooke. *SUS: a „quick and dirty“ usability scale*. 1996. Dostupné z <https://www.researchgate.net/publication/319394819>.
- [9] Milan Cifra. *Prieskumník sémantiky logiky prvého rádu*. Bakalárska práca, Univerzita Komenského v Bratislave. 2018. Dostupné z <https://opac.crzp.sk/?fn=detailBiblioForm&sid=65CF89236B0A7E124CD77AE040F7>.
- [10] Jozef Filip. *Reimplementácia prieskumníka štruktúr pre logiku prvého rádu*. Bakalárska práca, Univerzita Komenského v Bratislave. 2025. Dostupné z <https://opac.crzp.sk/?fn=detailBiblioForm&sid=8D80AD18777FCC7F64B6B74DC29E>.
- [11] Norbert Jurík. *Webový editor rezolvenčných dôkazov*. Bakalárska práca, Univerzita Komenského v Bratislave. 2020. Dostupné z <https://opac.crzp.sk/?fn=detailBiblioForm&sid=69B07E5EF4F2B22F0B84B4C7FD1B>.
- [12] Jozef Šiška, Ján Kluka, Ján Mazák. Prednášky z logiky pre informatikov. Dostupné z <https://fmfi-uk-1-ain-412.github.io/lpi/prednasky/poznamky-z-prednasok.pdf>.

- [13] Matej Mok. *Interaktívny pracovný zošit pre výučbu logiky pre informatikov. Bakalárska práca, Univerzita Komenského v Bratislave*. 2022. Dostupné z <https://opac.crzp.sk/?fn=detailBiblioForm&sid=72A4E145D65F5DA0BF4D3E0B29C0>.

Príloha A: Zadanie domácej úlohy

Úloha 6.2

📖 Zbierka: [príklad 7.2.2.](#)

Majme teóriu $T = \{A_1, \dots, A_3\}$ v jazyku $\mathcal{L}$ logiky prvého rádu s $\mathcal{C}_{\mathcal{L}} = \{a\}$, $\mathcal{F}_{\mathcal{L}} = \{f^1\}$, $\mathcal{P}_{\mathcal{L}} = \{A^1, B^1, C^2\}$, kde:

$$\begin{aligned}A_1 &= (\forall x (\neg A(f(x)) \wedge B(x)) \rightarrow \forall z C(a, z)), \\A_2 &= (\forall x \exists y C(x, y) \vee \forall x \neg (B(x) \rightarrow A(f(x)))), \\A_3 &= \forall x \forall y (C(x, y) \rightarrow \exists z (C(x, z) \vee C(y, z))).\end{aligned}$$

Upravte T na ekvisplniteľnú klauzálnu teóriu $T' = \bigcup_{i=1}^3 T'_i$ vo vhodnom rozšírení $\mathcal{L}'$ jazyka $\mathcal{L}$ o skolemovské konštanty a funkcie.

➔ Pokyny:

- Pre každú formulu A_i , $i = 1, \dots, 3$, zapíšte s ňou **ekvisplniteľnú množinu klauzúl** T'_i .
- Zapíšte aj **klúčové medzivýsledky** úprav (NNF, premenovanie premenných, skolemizáciu, PNF). Medzivýsledok nemusíte písať, ak sa podstatne nelíši od pôvodnej formuly alebo predchádzajúceho medzivýsledku.
- Nakoniec explicitne zapíšte **rozšírený jazyk** $\mathcal{L}'$.

✂ Svoj postup si môžete overiť pomocou nového [kontrolóra úprav do CNF](#).

- Predpripravený jazyk a formuly naimportujte zo súboru [du06.2.json](#).
- **Ak kontrolór využijete**, úpravy exportujte, premenujte na [du06.2.json](#), nahrajte do priečinka [teoreticke-ain](#) vo vetve [du06](#) vášho repozitára [lp126-«uniba-login»](#) a zaškrtnite [x] nižšie:

☐ Nahral/-a som úpravy do súboru [du06.2.json](#).

V tomto prípade nemusíte dokumentovať medzivýsledky, ale stále je potrebné zapísať jednotlivé čiastkové klauzálné teórie $T'_1, \dots, T'_3$.

Obr. 5.3: Zadanie domácej úlohy

Príloha B: Testovací dotazník

Kontrolór ekvivalentných úprav

Volám sa Jakub Šimon Štofanič a témou mojej bakalárskej práce je implementácia kontrolóra ekvivalentných úprav. Prosím vás o vyplnenie tohto dotazníka. Vaša spätná väzba pomôže pri usúdení užitočnosti a vylepšení kontrolóra. Vopred ďakujem za odpovede.

* Indicates required question

Na akom zariadení ste kontrolór použili? *

- ☐ Desktop
- ☐ Notebook
- ☐ Notebook s externým monitorom
- ☐ Mobil
- ☐ Tablet
- ☐ Other:

Na aký účel ste kontrolór použili? *

- ☐ Riešenie cvičenia
- ☐ Riešenie domácej úlohy
- ☐ Precvičovanie iných úloh
- ☐ Overenie riešenia, ktoré som vypracoval(a) mimo kontrolóra
- ☐ Other:

Potreboval(a) som sa naučiť mnoho vecí predtým, ako som mohol(a) začať pracovať s kontrolórom. *

1 2 3 4 5

Vôbec nesúhlasím ☐ ☐ ☐ ☐ ☐ Úplne súhlasím

Čo sa vám na kontrolóri páčilo?

Your answer

Čo sa vám na kontrolóri nepáčilo, chýbalo, čo by ste zmenili?

Your answer

Vaše univerzitné používateľské meno

Ako poďakovanie za vyplnenie dotazníka získate z predmetu Logika pre informatikov 1 prémiový bod. Na to je potrebné, aby ste sa identifikovali svojim univerzitným používateľským menom.

Ďalší prémiový bod získate, ak odovzdáte 6. domácu úlohu s exportom z kontrolóra ekvivalentných úprav (samozrejme, samostatne vypracovaným). Detaily nájdete v zadaní úlohy 6.2.

Your answer

Submit

[Clear form](#)

Never submit passwords through Google Forms.

This content is neither created nor endorsed by Google. - [Contact form owner](#) - [Terms of Service](#) - [Privacy Policy](#)

Does this form look suspicious? [Report](#)

Google Forms